

Bansilal Ramnath Agarwal Charitable Trust's
Vishwakarma Institute of Technology, Pune-37

(An autonomous Institute of Savitribai Phule Pune University)



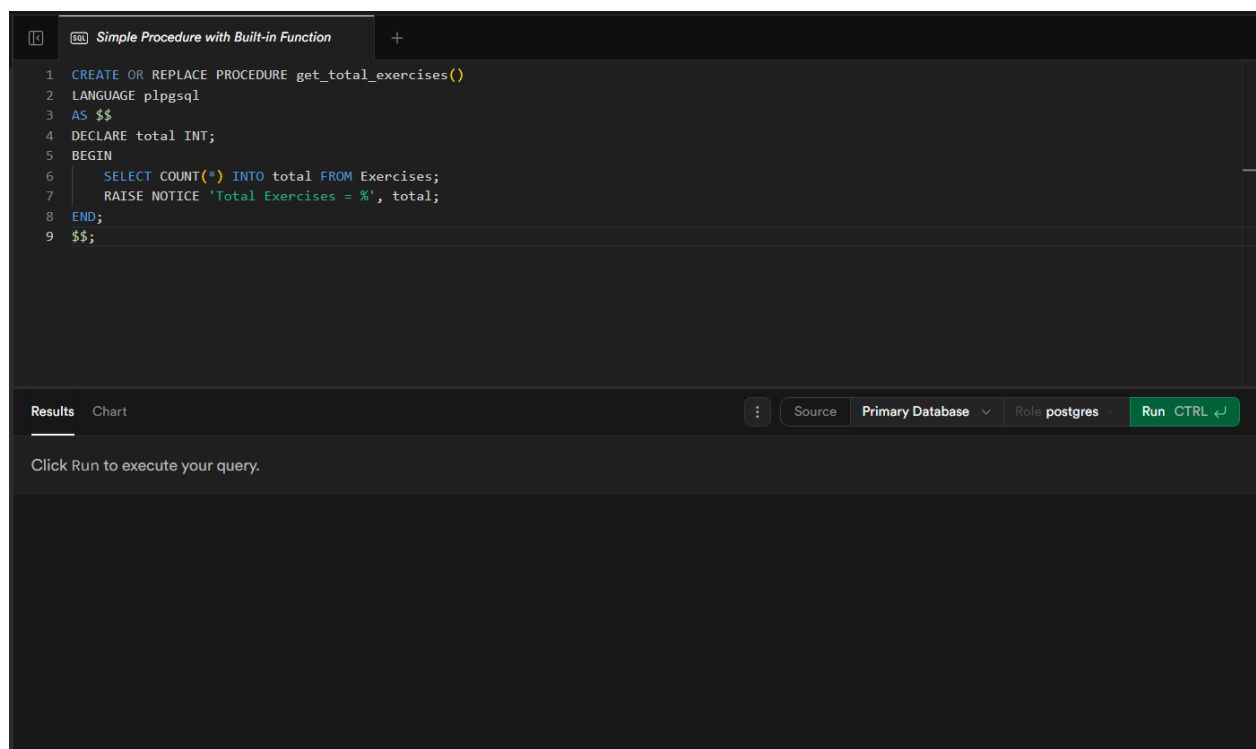
Department of Multidisciplinary
Engineering

Division	AIML-A
Batch	01
GR No.	12414243
Roll No.	08
Name	Abhishek Anil Patil

Title:

Write useful stored procedures in PL/SQL.

Description: In this task, we used a built-in function inside a stored procedure to perform an automatic calculation on the data. The function used was COUNT(), which is an aggregate function that returns the total number of records in a table. It was applied to the Exercises table to find how many exercises are stored. This demonstrated how built-in functions can be used inside procedures to process and retrieve summarized data efficiently.

Screenshots/Output:

The screenshot displays a SQL IDE interface. The top panel shows a tab titled "Simple Procedure with Built-in Function". The main editor contains the following PL/SQL code:

```
1 CREATE OR REPLACE PROCEDURE get_total_exercises()  
2 LANGUAGE plpgsql  
3 AS $$  
4 DECLARE total INT;  
5 BEGIN  
6     SELECT COUNT(*) INTO total FROM Exercises;  
7     RAISE NOTICE 'Total Exercises = %', total;  
8 END;  
9 $$;
```

Below the editor, there is a "Results" tab and a "Chart" tab. To the right of these tabs are buttons for "Source", "Primary Database", "Role postgres", and a green "Run" button with a keyboard shortcut "CTRL ↵". Below these buttons, a message reads "Click Run to execute your query."

10

SQL

Nested Query Procedure with LIMIT Clause

+

```
1 CREATE OR REPLACE PROCEDURE get_first_exercise()
2 LANGUAGE plpgsql
3 AS $$
4 DECLARE ex_name TEXT;
5 BEGIN
6     SELECT exercise_name
7     INTO ex_name
8     FROM Exercises
9     WHERE equipment_id = (
10         SELECT MIN(equipment_id) FROM Exercises
11     )
12     LIMIT 1;
13     RAISE NOTICE 'First Exercise = %', ex_name;
14 END;
15 $$;
```

Results Chart

⋮ Source Primary Database Role postgres Run CTRL ↵

Click Run to execute your query.

10

SQL

Correlated Query Procedure

+

```
1 CREATE OR REPLACE PROCEDURE get_valid_exercises()
2 LANGUAGE plpgsql
3 AS $$
4 DECLARE rec RECORD;
5 BEGIN
6     FOR rec IN
7         SELECT e.exercise_name, e.equipment_id
8         FROM Exercises e
9         WHERE EXISTS (
10             SELECT 1
11             FROM Available_Equipments eq
12             WHERE eq.equipment_id = e.equipment_id
13         )
14     LOOP
15         RAISE NOTICE 'Exercise: %, Equipment ID: %', rec.exercise_name, rec.equipment_id;
16     END LOOP;
17 END;
18 $$;
```

Results Chart

⋮ Source Primary Database Role postgres Run CTRL ↵

Click Run to execute your query.

SQL

Parameter Passing Procedure

+

```
1 CREATE OR REPLACE PROCEDURE get_exercises_by_muscle(p_muscle TEXT)
2 LANGUAGE plpgsql
3 AS $$
4 DECLARE rec RECORD;
5 BEGIN
6     FOR rec IN
7         SELECT exercise_name
8         FROM Exercises
9         WHERE muscle_group = p_muscle
10    LOOP
11        RAISE NOTICE 'Exercise: %', rec.exercise_name;
12    END LOOP;
13 END;
14 $$;
15 |
```

Results

Chart

⋮

Source

Primary Database

Role postgres

Run CTRL ↵

Click Run to execute your query.

SQL

Procedure Returning Some Value

+

```
1 CREATE OR REPLACE FUNCTION get_total equipments()
2 RETURNS INT
3 LANGUAGE plpgsql
4 AS $$
5 DECLARE total INT;
6 BEGIN
7     SELECT COUNT(*) INTO total FROM Available_Equipments;
8     RETURN total;
9 END;
10 $$;
11 |
```

Results

Chart

⋮

Source

Primary Database

Role postgres

Run CTRL ↵

Click Run to execute your query.